

HelixConstitution

HelixConstitution

The universal engineering constitution every project inherits — anti-bluff law, enforced mechanically, shared as one Git submodule.

Summary

HelixConstitution is the single, project-agnostic rulebook — added as a Git submodule by every Helix/vasic-digital project — that encodes non-negotiable engineering discipline (anti-bluff, evidence-only validation, data/host safety, documentation and test coverage) and propagates it to a fleet of 140+ repositories. It is the governance backbone that makes the whole family coherent.

Short description

A universal, inheritable Constitution shipped as a Git submodule. It defines mandatory, non-negotiable rules — anti-bluff evidence gates, false-positive immunity, data and host safety, coverage and documentation discipline — that every consuming project inherits automatically and may extend but never weaken.

Long description

HelixConstitution is the canonical, single source of truth for the engineering practices shared across every project that opts in by adding it as a Git submodule — engineering law, distributed and version-pinned exactly like code. Its centrepiece — `Constitution.md` — is an ~1 MB, continuously-versioned document of numbered clauses (the §11.4.x covenant family, currently through §11.4.170) plus per-agent operating manuals (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`) that import it by reference so that humans and every CLI agent read from one identical rulebook. Inheritance is deliberately three-layer: the universal base (this submodule), the project layer (a project's own `Constitution/CLAUDE/AGENTS` that

extend it), and an optional per-subdirectory layer — evaluated top-to-bottom, where a project may *tighten* the rules but is architecturally forbidden from *weakening* them. The result is a fleet of 140+ repositories that cannot silently drift apart, because the discipline they share is pinned, not remembered.

The document is uncompromisingly domain-agnostic: anything naming a specific vendor, hardware SKU, port, or library version must move down into the consuming project's own Constitution, and universality is never assumed — it has to be *earned* against an explicit four-part test before a rule is allowed into the base. Its philosophical spine is anti-bluff, expressed as an interlocking family of covenants — §1.1 false-positive immunity, §11.4 end-user quality covenant, §11.4.6 no-guessing, §11.4.69 positive-evidence taxonomy — whose combined effect is a single hard line: the bar for shipping is never “tests pass,” it is “a real user can use the feature,” and every green result must cite captured physical evidence or it does not count. A companion `submodules-catalogue.md` (142 repos) turns “do we already own something that does this?” into a catalogue-first, extend-don't-reimplement reflex before a line of new code is written. Helper scripts locate the submodule from any nesting depth and fan every commit out to four independent Git providers, so the one authoritative rulebook is also impossible to lose.

Why we built it

Multiple large product apps and dozens of decoupled reusable submodules, authored by the same owner, kept re-deriving the same hard-won rules — and kept hitting the same failure class: tests and status reports that claim success while the feature is broken for the end user (“PASS-bluffs” and “FAIL-bluffs”). Each forensic anchor in the Constitution records a real incident (e.g. the 2026-05-20 D3 audio-routing PASS-bluff where validation went green with an empty “Codec In Use” field, or the 2026-06-25 giant-button UI that passed token-equality tests while the real screen was broken). The Constitution exists to make that whole class of dishonest success mechanically impossible, once, universally — so the discipline cannot drift between projects or be quietly forgotten.

Why it's a game-changer

It converts engineering culture from documentation-people-hope-to-follow into inherited, versioned, mechanically-enforced law — the difference between a style guide and a compiler. One submodule bump upgrades the rules for the entire fleet at once, atomically and traceably. A single anti-bluff covenant is *guaranteed* present in every consuming repo, not by trust but by

construction: a propagation gate literally greps for the clause number across the fleet, and a paired mutation test proves the gate itself is not bluffing — so even the enforcement is enforced. Governance stops being an aspiration on a wiki nobody reads and becomes an auditable, testable fact you can point a CI job at.

What's innovative

- **Constitution-as-submodule** — engineering law distributed and version-pinned exactly like code, with deliberate v1.0.0-style tags and per-project pinning, so every repo knows *exactly* which revision of the law it is bound to.
- **Anti-bluff as a first-class, forensic doctrine** — every clause traces back to a verbatim operator mandate and, often, the exact real-world incident that motivated it, so the rulebook reads as case law rather than opinion.
- **Meta-testing of the rules themselves (§1.1)** — every gate is paired with a mutation that must flip PASS→FAIL, so “the gate isn’t a sham” is not asserted but proven on every run; a gate that can never fail is treated as worse than no gate at all.
- **Earned universality** — an explicit four-part test decides whether a rule is truly universal or merely project-specific, keeping the base lean, portable, and free of vendor leakage.

How it is used across all products (the powers it gives)

As a **mandatory governance pillar**, HelixConstitution is not a document the family consults — it is the load-bearing structure the family is built on:

- **Governance backbone:** every Helix/vasic-digital project adds it as a submodule and imports it from `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / its own `Constitution.md`; the rules apply unconditionally, from the first commit, with no per-project opt-out.
- **Gates & mandates:** it defines the four-layer coverage model — source-present, survives-build, behaves-at-runtime, gate-not-bluffing — that a feature must clear on all four levels before it counts as done, plus a growing roster of named mandates: credentials handling (§11.4.10), documentation always-sync (§11.4.60), containers-submodule mandate (§11.4.76), CodeGraph (§11.4.78), mandatory test-type coverage (§11.4.169), and more.
- **Propagation:** CM-COVENANT-114-NNN-PROPAGATION gates assert the *literal* clause text is present across the consuming fleet, so a covenant cannot be quietly dropped in one corner of the

estate; non-compliance is a hard release blocker with no escape-hatch flags to wave it through.

- **Discovery:** `submodules-catalogue.md` turns “do we already own something that does X?” into a one-glance answer before any new module is scaffolded, killing duplicate effort at the source.
- **Consistency of AI agents:** the same law is expressed identically to every CLI agent (Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi via `AGENTS.md`, Qwen Code via `QWEN.md`), so no matter which tool touches the code, it obeys one and the same covenant.

Biggest technical challenges & how we solved them

- **Locating the submodule from arbitrary nested depth** — a rule buried three submodules deep still has to find the law without knowing where it lives → `find_constitution.sh` walks up parent directories and follows the git superproject pointer recursively, honouring a `CONSTITUTION_DIR` override and two supported layouts (`constitution/`, `submodules/constitution/`), so resolution is deterministic no matter how deep the nesting goes.
- **Keeping one repo authoritative across four Git providers** — mirrors are worthless if they drift → `install_upstreams.sh` reads declarative `Upstreams/*.sh` remotes and configures origin with multiple push URLs, so a single `git push` fans out atomically to GitHub (primary), GitLab, GitFlic, and GitVerse and no mirror can fall behind.
- **Preventing rule-bloat / project leakage into the universal base** — every tempting “just add it here” erodes portability → the earned-universality four-part test plus §11.4.17 universal-vs-project classification is applied to *every* new rule, forcing project-specific concerns back down into the project layer where they belong.
- **Proving the inheritance gate actually works** — a gate you never see fail is a gate you can’t trust → `meta_test_inheritance.sh`, a sentinel meta-test, deliberately deletes the §11.4 anchor and asserts the gate catches it, so the enforcement mechanism itself is continuously re-verified against silent breakage.

Tech stack

- **Git-submodule inheritance** — *why*: Git submodules are the one mechanism that lets a rulebook be authoritative *and* version-pinned per consumer, upgraded by an explicit, reviewable bump rather than a silent copy-paste; *how*: consuming projects add the submodule and `@import` its agent

files, and the three layers are evaluated top-to-bottom with a strict extends-not-weakens contract at every boundary.

- **find_constitution.sh** — *why*: the rules are useless if deeply-nested code can't reliably find them, and hardcoding paths would break the moment a project reorganised; *how*: a parent-directory walk plus `git rev-parse --show-superproject-working-tree` recursion, backstopped by a `CONSTITUTION_DIR` override, resolving both supported layouts.
- **install_upstreams.sh + Upstreams/** — *why*: four-provider redundancy is only real if it takes zero extra effort to maintain, otherwise mirrors rot; *how*: declarative per-remote `.sh` files are materialised into a single multi-URL origin, collapsing four pushes into one.
- **§1.1 mutation meta-tests** — *why*: a gate that can never fail is worse than none because it manufactures false confidence; *how*: each gate is paired with a sed-out/rename mutation that must turn `PASS`→`FAIL` and is then restored, so every gate proves it still bites on every run.
- **Propagation gates (CM-COVENANT-114-NNN-PROPAGATION)** — *why*: a covenant is only universal if it is verifiably present in *every* consumer, not just the flagship repo; *how*: a literal clause-number `grep` across consumers, backed by a paired §1.1 mutation that proves the propagation check itself can fail.
- **submodules-catalogue.md (§11.4.74)** — *why*: the fastest way to violate anti-duplication discipline is to not know what you already own; *how*: a 142-repo, capability-grouped inventory, with a catalogue-check recorded in the tracker *before* anything new is scaffolded.
- **Multi-format export** — *why*: the same law must be equally consumable by humans reading it, tooling parsing it, and archives preserving it; *how*: every canonical doc is emitted as `.md` / `.html` / `.pdf` / `.docx` from one source.

Status & honesty notes

- **Status: shipped.** Actively versioned and in use as a submodule across the fleet (public canonical and mirror repos).
- **License: TBD** — not explicitly stated in the source material reviewed; confirm against the repository `LICENSE` before publishing.
- Additional upstream mirrors: GitLab `helixdevelopment1/helixconstitution`, GitFlic `helixdevelopment/helixconstitution`, GitVerse `helixdevelopment/HelixConstitution`.

Priority tier: Helix-primary — a mandatory governance pillar of how everything in the Helix family is built.